

L'observable Angular

Explication

Un flux de données est constitué de données qui arrivent sur un certain temps. Le flux de données peut être n'importe quoi. Comme les variables, les entrées utilisateur, les propriétés, les caches, les structures de données et même les échecs, etc.

À tout moment, le flux peut émettre les trois choses suivantes.

- value : c'est-à-dire la valeur suivante dans le flux
- complete : le flux est terminé
- error : L'erreur a arrêté le flux.

La programmation réactive consiste à créer le flux, à émettre des signaux value, error, ou complete, à manipuler, à transférer ou à faire quelque chose d'utile avec les flux de données. C'est là que les RxJ entrent en scène.

La **RxJS** (Reactive Extensions Library for JavaScript) est une bibliothèque Javascript qui nous permet de travailler avec des flux de données asynchrones.

Angular utilise largement la bibliothèque RxJS dans son Framework pour implémenter la programmation réactive dans plusieurs cas notamment les requêtes Http.

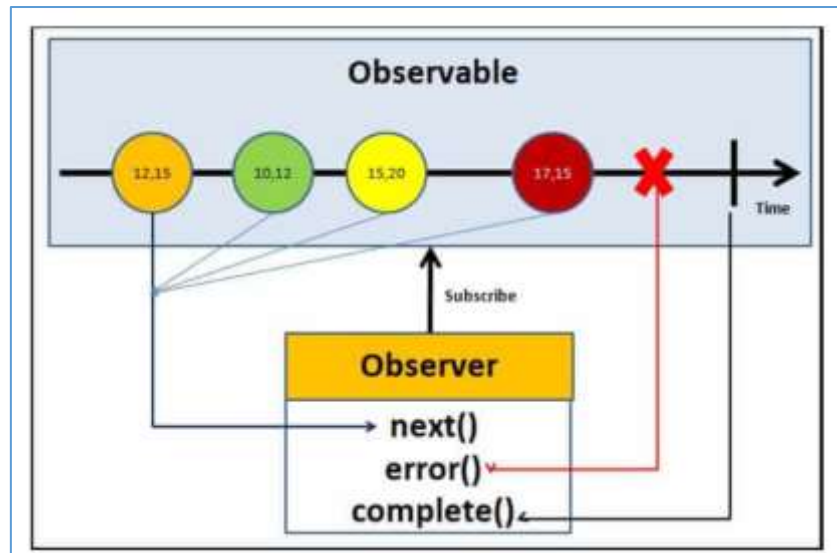
Observable est une fonction qui convertit le flux de données ordinaire en un flux observable. Vous pouvez considérer Observable comme un wrapper autour du flux de données ordinaire.

Un flux observable ou simple Observable émet la valeur du flux de manière asynchrone. Il émet les signaux complets lorsque le flux se termine ou un signal d'erreur en cas d'erreur du flux.

Les observables sont déclaratifs. Vous définissez une fonction observable comme n'importe quelle autre variable. L'observable ne commence à émettre des valeurs que lorsque quelqu'un s'y abonne.

L'Observable n'est utile que si quelqu'un consomme la valeur émise par l'observable. Nous les appelons **observateurs (observer)** ou abonnés.

Les observateurs communiquent avec l'Observable à l'aide de rappels. L'observateur doit s'abonner à l'observable pour recevoir la valeur de l'observable. Lors de l'abonnement, il transmet éventuellement les trois rappels. next(), error() et complete()



L'observable émet la valeur dès que l'observateur ou le consommateur y souscrit.

L'observable appelle le rappel `next()` chaque fois que la valeur arrive dans le flux. Il transmet la valeur comme argument au prochain rappel. Si l'erreur se produit, le rappel `error()` est invoqué. Il appelle le rappel `complete()` lorsque le flux se termine.

Les observateurs/abonnés s'abonnent à Observables.

L'observateur enregistre trois rappels auprès de l'observable au moment de l'abonnement. c'est-à-dire `next()`, `error()` et `complete()`

Les trois rappels sont facultatifs.

L'observateur reçoit les données de l'observable via le rappel `next()`

Ils reçoivent également les erreurs et les événements d'achèvement de l'Observable via les rappels `error()` et `complete()`

Etapes de réalisation

```
ng g c stream/exemple-observable
```

```
src > app > stream > exemple-observable >  exemple-observable.component.html >
```

```
<h2> Exemple pratique sur l'observable Angular </h2>
<ol>
  <li *ngFor="let log of progress">{{ log }}</li>
</ol>
```

```
src > app > stream > exemple-observable >  exemple-observable.component.ts >
```

```
import { Component } from '@angular/core';
import { Observable } from 'rxjs';
```

```

@Component({
  selector: 'app-exemple-observable',
  templateUrl: './exemple-observable.component.html',
  styleUrls: ['./exemple-observable.component.css']
})
export class ExempleObservableComponent {
  progress: Array<string> = [];

  ngOnInit() {
    this.loadData().subscribe({
      next:(x:any) => this.progress.push(x),
      error: (error) => this.progress.push(error),
      complete: () => this.progress.push('>>>>> Complete')
    });
  }

  loadData() {
    return new Observable(observer => {
      observer.next('Observer work 1');
      observer.next('Observer work 2');
      setTimeout(() => {
        observer.next('Observer work 3');
        observer.complete();
      }, 5000);
    });
  }
}

```

src > app >  app.component.html >

```
<app-exemple-observable></app-exemple-observable>
```

Instant 1

Exemple pratique sur l'observable Angular

1. Observer work 1
2. Observer work 2

Instant 2 (après 5s)

Exemple pratique sur l'observable Angular

1. Observer work 1
2. Observer work 2
3. Observer work 3
4. >>>> Complete